

Detailed Notes

Chapter 6 — Functions

Topics Covered

- Introduction to Functions
- The General Form of a Function
- Understanding the Scope of a Function
 - Function Arguments
 - Call by Value
 - Creating a Call by Reference
 - Calling Functions with Arrays
- argc and argv — Arguments to main()
 - The return Statement
 - Returning from a Function
 - Returning Values
 - Returning Pointers
 - Functions of Type void
 - What Does main() Return?
 - Recursion
 - Function Prototypes
 - Old-Style Function Declarations
 - Standard Library Function Prototypes
- Declaring Variable Length Parameter Lists
 - The Implicit int Rule
- Old-Style vs. Modern Parameter Declarations
 - The inline Keyword (C99)

CHAPTER 6 — FUNCTIONS

Functions are the building blocks of C and the place where all program activity occurs. This chapter examines their features, including function arguments, return values, prototypes, and recursion.

Key Feature	Description
Building blocks	All program activity in C occurs inside functions
Modular design	Functions allow code to be broken into discrete, reusable tasks
Arguments	Information is passed to functions via arguments (parameters)
Return values	Functions can return a value to the caller
Prototypes	Declarations that enable strong type checking by the compiler
Recursion	A function can call itself to solve problems defined in terms of themselves

THE GENERAL FORM OF A FUNCTION

The general form of a C function is:

```
ret-type function-name(parameter list)
{
    body of the function
}
```

Component	Description
ret-type	Specifies the type of data that the function returns. A function may return any type of data EXCEPT an array.
function-name	The name of the function — any valid C identifier
parameter list	A comma-separated list of variable names and their associated types. The parameters receive the values of the arguments when the function is called.
body	The statements that define what the function does, enclosed in { }

- A function can be without parameters — the parameter list is then empty or explicitly declared void
- An empty parameter list can be explicitly specified using the keyword void inside the parentheses

Important parameter declaration rule — unlike regular variable declarations (where multiple variables can share a type), ALL function parameters must be declared individually, each including both the type and the name:

```
f(int i, int k, int j)    /* CORRECT: each parameter has its own type */
f(int i, k, float j)     /* WRONG: k must have its own type specifier */
```

UNDERSTANDING THE SCOPE OF A FUNCTION

The scope rules of a language are the rules that govern whether a piece of code knows about or has access to another piece of code or data.

Scope Concept	Explanation
Function defines block scope	Each function is a discrete block of code. Its code is private to that function and cannot be accessed by any statement in any other function except through a call to that function.
No goto between functions	You cannot use goto to jump into the middle of another function.
Isolated code and data	Unless a function uses global variables, it can neither affect nor be affected by other parts of the program.
Local variables	Variables defined within a function are local variables. They come into existence when the function is entered and are destroyed upon exit.
Static exception	If a local variable is declared static, the compiler treats it as global for storage purposes but limits its scope to the function — it retains its value between calls.
Formal parameters	Parameters fall within the function's scope — known throughout the entire function. They come into existence when the function is called and are destroyed when it exits.
File scope for functions	All functions have file scope. Therefore, you CANNOT define a function within a function. This is why C is not technically a block-structured language.

FUNCTION ARGUMENTS

If a function is to accept arguments, it must declare the parameters that will receive the values of the arguments. Parameter declarations occur after the function name:

```
/* Return 1 if c is part of string s; 0 otherwise. */
int is_in(char *s, char c)
{
    while(*s)
        if(*s==c) return 1;
        else s++;
    return 0;
}
/* is_in() has two parameters: s and c.
   Returns 1 if character c is found anywhere in string s; else 0. */
```

- Parameters perform the special task of receiving the value of the arguments passed to the function
- Parameters behave like any other local variable — you can make assignments to them or use them in expressions

Call by Value

In a computer language there are two ways that arguments can be passed to a subroutine:

Method	How It Works	Effect on Calling Argument
Call by Value	Copies the value of the argument into the formal parameter. Changes made to the parameter have NO effect on the original argument.	Argument is unchanged after the call
Call by Reference	Copies the ADDRESS of the argument into the parameter. Code inside the subroutine uses the address to access and modify the actual argument.	Argument CAN be changed by the function

With few exceptions, C uses call by value to pass arguments. In general, this means code within a function cannot alter the arguments used to call the function:

```
#include <stdio.h>
int sqr(int x);
int main(void)
{
    int t = 10;
    printf("%d %d", sqr(t), t);
    return 0;
}
int sqr(int x)
{
    x = x*x;    /* only the LOCAL copy of x is modified */
    return(x);
}
/* Output: 100 10
The variable t in main() still has the value 10.
Only the local parameter x was changed inside sqr(). */
```

NOTE It is a *COPY* of the value of the argument that is passed into a function. What occurs inside the function has no effect on the variable used in the call.

Creating a Call by Reference

Even though C uses call by value, you can create a call by reference by passing a **POINTER** to an argument instead of passing the argument itself. Since the address of the argument is passed, code within the function can change the value of the argument outside the function.

Example — the `swap()` function exchanges the values of two integer variables using pointer parameters:

```
void swap(int *x, int *y)
{
    int temp;
    temp = *x;    /* save the value at address x */
    *x = *y;      /* put y into x */
    *y = temp;    /* put x into y */
}
```

`swap()` must be called with the ADDRESSES of the arguments (using the `&` operator):

```
#include <stdio.h>
void swap(int *x, int *y);
int main(void)
{
    int i = 10, j = 20;
    printf("i and j before swapping: %d %d\n", i, j);
    swap(&i, &j); /* pass the ADDRESSES of i and j */
    printf("i and j after swapping: %d %d\n", i, j);
    return 0;
}
/* Output:
i and j before swapping: 10 20
i and j after swapping: 20 10 */
```

- The unary `&` operator is used to produce the address of the variables
- The addresses of `i` and `j` — not their values — are passed into `swap()`
- Inside `swap()`, the contents of the variables are accessed using standard pointer operations (`*x`, `*y`)

NOTE C++ allows you to fully automate a call by reference through the use of reference parameters. Reference parameters are NOT supported by C.

Calling Functions with Arrays

When an array is used as a function argument, its ADDRESS is passed to the function. This is an exception to the normal call-by-value parameter passing convention. The code inside the function operates on — and potentially alters — the actual contents of the array used in the call.

Example — `print_upper()` converts and prints a string in uppercase, and MODIFIES the original array:

```
#include <stdio.h>
#include <ctype.h>
void print_upper(char *string);
int main(void)
{
    char s[80];
    printf("Enter a string: ");
    gets(s);
    print_upper(s);
    printf("\ns is now uppercase: %s", s);
    return 0;
}
void print_upper(char *string)
{
    register int t;
    for(t=0; string[t]; ++t) {
        string[t] = toupper(string[t]);
        putchar(string[t]);
    }
}
/* Sample output:
Enter a string: This is a test.
THIS IS A TEST.
s is now uppercase: THIS IS A TEST. */
```

If you do NOT want the original array modified, do NOT alter `string[t]` inside the function — only read from it:

```
void print_upper(char *string)
{
    register int t;
    for(t=0; string[t]; ++t)
        putchar(toupper(string[t]));    /* does NOT write back to string[] */
}
/* s in main() remains unchanged because no writes are made to string[] */
```

Example of an array-passing pattern — `xgets()`, a simplified version of the standard `gets()` function:

```
/* A simple version of the standard gets() library function. */
char *xgets(char *s)
{
    char ch, *p;
    int t;
    p = s;    /* gets() returns a pointer to s */
    for(t=0; t<80; ++t) {
        ch = getchar();
        switch(ch) {
            case '\n':
                s[t] = '\0';    /* null-terminate the string */
                return p;
            case '\b':
                if(t>0) t--;    /* backspace: remove last character */
                break;
            default:
                s[t] = ch;    /* store character in array */
        }
    }
    s[79] = '\0';
    return p;
}
/* xgets() must be called with a char * pointer (e.g., a char array name).
   The loop is limited to 80 chars to prevent buffer overrun.
   On return, the array used to call xgets() contains the typed string. */
```

argc AND argv — ARGUMENTS TO `main()`

Sometimes it is useful to pass information into a program when you run it. The information is passed via command line arguments — the information that follows the program's name on the operating system command line.

For example, when compiling a program you might type:

```
cc program_name
```

Here, `program_name` is a command line argument.

Parameter	Type	Description
argc	int	Holds the NUMBER of arguments on the command line. Always at least 1 because the program's name is always the first argument.
argv	char *argv[]	A POINTER TO AN ARRAY of character pointers. Each element in this array points to a command line argument string. argv[0] = program name, argv[1] = first argument, etc.

- All command line arguments are strings — any numbers must be converted to binary format by the program manually (e.g., using atoi())
- In many environments, each command line argument must be separated by a space or a tab
- Commas, semicolons, and similar characters are NOT considered separators
- Some environments allow strings containing spaces to be enclosed in double quotes — the entire quoted string is treated as a single argument

The declaration of main() with command line parameters:

```
int main(int argc, char *argv[])
{
    /* ... */
}
```

Example 1 — Hello program: prints Hello and your name from the command line:

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[])
{
    if(argc != 2) {
        printf("You forgot to type your name.\n");
        exit(1);
    }
    printf("Hello %s", argv[1]);
    return 0;
}
/* If program is called 'name' and user types: name Tom
   Output: Hello Tom
   argv[0] = "name", argv[1] = "Tom" */
```

Example 2 — Countdown program: counts down from a specified number; displays if 'display' is passed as second argument:

```
/* Countdown program. */
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>
int main(int argc, char *argv[])
{
    int disp, count;
    if(argc < 2) {
        printf("You must enter the length of the count\n");
        printf("on the command line. Try again.\n");
        exit(1);
    }
}
```

```
    if(argc==3 && !strcmp(argv[2], "display")) disp = 1;
    else disp = 0;
    for(count=atoi(argv[1]); count; --count)
        if(disp) printf("%d\n", count);
    putchar('\a'); /* ring the bell */
    printf("Done");
    return 0;
}
/* Usage: countdown 10          -- counts down silently from 10
   countdown 10 display -- displays count and rings bell */
```

Example 3 — Accessing individual characters in command line arguments using a second index into argv:

```
#include <stdio.h>
int main(int argc, char *argv[])
{
    int t, i;
    for(t=0; t<argc; ++t) {
        i = 0;
        while(argv[t][i]) {
            putchar(argv[t][i]);
            ++i;
        }
        printf("\n");
    }
    return 0;
}
/* argv[t] accesses the t-th argument string */
/* argv[t][i] accesses the i-th character of the t-th argument */
```

Key points about argc and argv:

- argc and argv are traditional names — you may name these two parameters to main() anything you like
- Some compilers may support additional arguments to main() — check your compiler's documentation
- When a program does not require command line parameters, declare main() as: int main(void)
- Using command line arguments gives your program a professional appearance and facilitates use in batch files and scripts

THE return STATEMENT

The return statement has two important uses:

- 1. It causes an immediate exit from the function — program execution returns to the calling code
- 2. It can be used to return a value to the caller

Returning from a Function

A function terminates and returns to the caller in two ways:

Method	Description
Fall through (implicit)	Execution of the last statement in the function, whereupon the function's closing } is conceptually encountered. No value is returned explicitly.
return statement (explicit)	Most functions use an explicit return statement to stop execution — either because a value must be returned or to make the code simpler and more efficient.

Example — `pr_reverse()` prints a string backwards then returns implicitly:

```
#include <string.h>
#include <stdio.h>
void pr_reverse(char *s);
int main(void)
{
    pr_reverse("I like C");
    return 0;
}
void pr_reverse(char *s)
{
    register int t;
    for(t=strlen(s)-1; t>=0; t--) putchar(s[t]);
    /* no return statement: function returns implicitly when loop ends */
}
```

A function may contain SEVERAL return statements. Example — `find_substr()` uses two returns:

```
#include <stdio.h>
int find_substr(char *s1, char *s2);
int main(void)
{
    if(find_substr("C is fun", "is") != -1)
        printf("Substring is found.");
    return 0;
}
/* Return index of first match of s2 in s1; -1 if not found. */
int find_substr(char *s1, char *s2)
{
    register int t;
    char *p, *p2;
    for(t=0; s1[t]; t++) {
        p = &s1[t];
        p2 = s2;
        while(*p2 && *p2==*p) { p++; p2++; }
        if(!*p2) return t; /* 1st return: match found at index t */
    }
    return -1; /* 2nd return: no match found */
}
```

Returning Values

All functions, except those of type void, return a value. This value is specified by the return statement.

Standard	Rule for Return Values
C89	If a non-void function executes a return statement with no value, a garbage value is returned — bad practice but not a syntax error.

Standard	Rule for Return Values
C99	A non-void function MUST use a return statement that includes a value. If execution reaches the closing }, a garbage value is returned — still a fundamental flaw but not a syntax error.
C++	Same as C99 — a non-void function must return a value.

As long as a function is not declared void, it can be used as an operand in an expression:

```
x = power(y);                /* assign return value */
if(max(x,y) > 100) printf("greater"); /* use in a condition */
for(ch=getchar(); isdigit(ch); ) ...; /* use in a loop control */
```

A function call cannot appear on the **LEFT** side of an assignment:

```
swap(x,y) = 100; /* INCORRECT — compiler will flag as error */
```

The three types of functions in C:

Type	Description	Examples
Computational (pure function)	Performs operations on arguments and returns a computed value	sqrt(), sin() — compute square root, sine
Manipulative (status function)	Manipulates information and returns success/failure indication	fclose() — returns 0 for success, EOF for error
Procedural (void function)	Strictly procedural — produces no meaningful return value. Should be declared void to prevent accidental misuse.	exit() — terminates the program

The return value of a function does not **HAVE** to be used — it can be simply discarded:

```
#include <stdio.h>
int mul(int a, int b);
int main(void)
{
    int x=10, y=20, z;
    z = mul(x, y); /* 1: return value assigned to z */
    printf("%d", mul(x,y)); /* 2: return value used by printf() */
    mul(x, y); /* 3: return value is lost (discarded) */
    return 0;
}
int mul(int a, int b) { return a*b; }
/* All three calls are valid. In line 3, the return value is simply ignored. */
```

Returning Pointers

Functions that return pointers are handled just like any other type of function — but there are key rules:

- Pointers are neither integers nor unsigned integers — they are memory addresses of a certain type of data

- Pointer arithmetic is relative to the base type — an `int *` incremented points 4 bytes forward (not 1)
- A function that returns a pointer must explicitly declare what TYPE of pointer it is returning
- Do NOT use a return type of `int *` to return a `char *` pointer
- If a function needs to return a generic pointer, declare the return type as `void *`

To return a pointer, the function must be declared with a pointer return type:

```
/* Return pointer to first occurrence of c in string s.
   If no match, returns pointer to null terminator. */
char *match(char c, char *s)
{
    while(c!=*s && *s) s++;
    return(s);
}
```

Program using `match()`:

```
#include <stdio.h>
char *match(char c, char *s);
int main(void)
{
    char s[80], *p, ch;
    gets(s);
    ch = getchar();
    p = match(ch, s);
    if(*p) /* there is a match */
        printf("%s ", p);
    else
        printf("No match found.");
    return 0;
}
/* If ch is in s, p points to that character in s and the rest of the string prints.
   If no match, p points to the null terminator — *p is false (0). */
```

Functions of Type void

One of `void`'s uses is to explicitly declare functions that do not return values. This prevents their use in any expression and helps avert accidental misuse.

```
/* print_vertical() prints its string argument vertically down the screen. */
/* Declared void because it returns no useful value. */
void print_vertical(char *str)
{
    while(*str)
        printf("%c\n", *str++);
}
```

```
#include <stdio.h>
void print_vertical(char *str);
int main(int argc, char *argv[])
{
    if(argc > 1) print_vertical(argv[1]);
    return 0;
}
/* Usage: program Hello → prints H, e, l, l, o each on a new line */
```

NOTE *Early versions of C did not define the void keyword. Thus, in early C programs, functions that did not return values simply defaulted to type int, even though no value was returned.*

WHAT DOES main() RETURN?

The main() function returns an integer to the calling process — which is generally the operating system.

Return Value	Meaning
0	Conventional: normal, successful program termination
Non-zero	Conventional: some kind of error or abnormal termination
Undefined (no explicit return)	Technically the value passed to the OS is undefined. Most C compilers automatically return 0, but do NOT rely on this if portability is a concern.

- Returning a value from main() is equivalent to calling exit() with the same value
- The return value allows batch files and scripts to test whether a program succeeded or failed

RECURSION

In C, a function can call itself. A function that calls itself is said to be recursive. Recursion is the process of defining something in terms of itself — sometimes called circular definition.

Example — computing the factorial of a number using both recursive and iterative approaches:

```
/* Recursive version */
int factr(int n)
{
    int answer;
    if(n==1) return(1);
    answer = factr(n-1) * n;    /* recursive call */
    return(answer);
}

/* Non-recursive (iterative) version */
int fact(int n)
{
    int t, answer;
    answer = 1;
    for(t=1; t<=n; t++)
        answer = answer * t;
    return(answer);
}
```

Trace of factr(3):

Call	n	Action	Returns
factr(3)	3	answer = factr(2) * 3 — calls factr(2) first	6
factr(2)	2	answer = factr(1) * 2 — calls factr(1) first	2
factr(1)	1	Base case: returns 1 immediately	1

How recursion works in memory:

- When a function calls itself, a NEW SET of local variables and parameters are allocated storage on the stack
- A recursive call does NOT make a new copy of the function code — only the values being operated on are new
- As each recursive call returns, the old local variables and parameters are removed from the stack
- Execution resumes immediately after the recursive call inside the calling function
- Recursive functions could be said to 'telescope' out and back

Advantages and disadvantages of recursion:

Aspect	Recursive	Iterative
Code clarity	Can be clearer and simpler for certain algorithms (e.g., quicksort, AI problems)	Sometimes harder to read for inherently recursive problems
Code size	Often similar to iterative — rarely offers significant reduction	Often similar
Memory use	Each call adds a new stack frame — can exhaust the stack for deep recursion	Typically uses less memory — no repeated stack frames
Speed	Often slightly slower due to overhead of repeated function calls	Often faster — no function call overhead
Stack risk	Deep recursion can cause stack overrun and crash the program	No risk of stack overrun from recursion

NOTE When writing recursive functions, you *MUST* have a conditional statement (such as an *if*) somewhere to force the function to return without the recursive call being executed. If you omit this base case, the function will never return once called — causing a stack overrun and crash. Omitting the conditional (base case) is the most common error when writing recursive functions.

Good practices for recursive programming:

- Always define a base case — a condition where the function returns directly without calling itself
- Use `printf()` liberally during development to watch the level of each call and intermediate values
- Consider whether an iterative solution would be clearer or more efficient for your specific problem

FUNCTION PROTOTYPES

In modern, properly written C programs, all functions must be declared before they are used. This is normally accomplished using a function prototype. Function prototypes were not part of the original C language — they were added by C89.

Feature	Detail
Required by C?	Not technically required, but strongly encouraged
Required by C++?	YES — prototypes are required by C++
Purpose	Enable the compiler to provide stronger type checking — similar to Pascal
What they catch	Questionable type conversions between arguments and parameter types; wrong number of arguments in a call

General form of a function prototype:

```
type func_name(type parm_name1, type parm_name2, ..., type parm_nameN);

/* Examples: */
int find_substr(char *s1, char *s2);
void swap(int *x, int *y);
double sqrt(double x);
```

- The use of parameter names is optional in a prototype
- Including parameter names is recommended — they allow the compiler to identify type mismatches by name

Example — prototype catches a type mismatch at compile time:

```
/* This program uses a function prototype to enforce strong type checking. */
void sqr_it(int *i); /* prototype: expects an int POINTER */
int main(void)
{
    int x = 10;
    sqr_it(x); /* type mismatch: passing int, not int * */
    return 0;
}
void sqr_it(int *i)
{
    *i = *i * *i;
}
/* The compiler will report an error: sqr_it() expects int *, received int */
```

When Can a Definition Serve as a Prototype?

A function's definition can also serve as its prototype if the definition occurs PRIOR to the function's first use in the program:

```
#include <stdio.h>
/* This definition also serves as the prototype — no separate prototype needed */
void f(int a, int b)
{
    printf("%d ", a % b);
}
int main(void)
```

```
{
    f(10, 3);
    return 0;
}
```

- While this works in small programs, it is rarely possible in large programs with multiple files
- All examples in this book include a SEPARATE prototype for each function — this is standard practice
- The only function that does NOT require a prototype is `main()` — it is the first function called when the program begins

Prototyping Functions with No Parameters

There is an important difference between C and C++ in how a function with no parameters is prototyped:

```
int f();          /* C++: prototype for a function with NO parameters */
                  /* C:   means 'no parameter information is given' - parameters may
                  exist! */

float f(void);    /* C: correct way to say 'this function has NO parameters' */
                  /* C++: still allowed but redundant */
```

Declaration	Meaning in C	Meaning in C++
<code>int f();</code>	Old-style: no parameter information given — function COULD have parameters	Prototype with NO parameters — calling with arguments is an error
<code>int f(void);</code>	Explicitly NO parameters — calling with arguments is an error	Redundant but allowed — same as <code>int f()</code>

Old-Style Function Declarations

In the early days of C, before prototypes existed, functions were declared using an old-style format that specified only the return type, without any parameter information:

```
double div();    /* old-style function declaration - NO parameter info */

/* General form: */
type_specifier function_name(); /* parameter list is empty */
```

Old-style declaration example:

```
#include <stdio.h>
double div(); /* old-style: tells compiler div() returns double */
int main(void)
{
    printf("%f", div(10.2, 20.0));
    return 0;
}
double div(double num, double denom)
{
    return num / denom;
}
```

```
/* The old-style declaration tells the compiler the return type only.
   It says nothing about the parameters. */
```

REMEMBER *The old-style function declaration is outmoded and should NOT be used for new code. It is also incompatible with C++.*

Standard Library Function Prototypes

Any standard library function used by your program must be prototyped. This is accomplished by including the appropriate header file for each library function:

Header File	Contains Prototypes For
<stdio.h>	printf(), scanf(), gets(), puts(), fopen(), fclose(), etc.
<string.h>	strcpy(), strcat(), strlen(), strcmp(), strchr(), strstr(), etc.
<math.h>	sqrt(), sin(), cos(), tan(), pow(), fabs(), etc.
<stdlib.h>	malloc(), free(), atoi(), atof(), exit(), abs(), etc.
<ctype.h>	isdigit(), isalpha(), toupper(), tolower(), isspace(), etc.

- A header contains two main elements: definitions used by the library functions, and the prototypes for those functions
- For example, <stdio.h> is included in almost all programs because it contains the prototype for printf()

DECLARING VARIABLE LENGTH PARAMETER LISTS

You can specify a function that has a variable number of parameters. The most common example is printf(). To tell the compiler that an unknown number of arguments will be passed to a function, end the declaration of its parameters using three periods (...):

```
int func(int a, int b, ...);
/* func() has at least two integer parameters (a and b)
   and an unknown number (including 0) of additional parameters after that */
```

- This form of declaration is used in both the prototype and the function's definition
- Any function that uses a variable number of parameters **MUST** have at least one actual named parameter

```
int func(...); /* ILLEGAL — no actual (named) parameter before ... */
```

The Implicit int Rule

The original version of C included the 'implicit int' rule (also called the 'default to int' rule): in the absence of an explicit type specifier, the type int is assumed.

Standard	Implicit int Rule
Original C (pre-C89)	Fully supported — widely used for function return types and parameters
C89	Included for compatibility with original C code
C99	ELIMINATED — the implicit int rule is NOT supported
C++	NOT supported

Most common use was the return type of functions:

```
/* Modern, correct form: */
int f(void)
{
    return 0;
}

/* Old implicit-int form (C89 and earlier): */
f(void) /* return type int by default — bad practice */
{
    return 0;
}
```

The implicit int rule applied to parameters and local variables too (C89 and earlier):

```
/* C89: return type, parameter types, and local variable type all default to int */
f(register a, register b)
{
    register c; /* c defaults to int */
    c = a + b;
    printf("%d", c);
    return c;
}
/* NOT recommended — do not use this style */
```

NOTE The implicit int rule is NOT supported by C99 or C++. Its use in C89-compatible programs is not recommended. Always explicitly specify every type used by your program.

Old-Style vs. Modern Function Parameter Declarations

Standard C supports two forms of function parameter declaration. The modern form is strongly recommended. C++ supports only the modern form.

Form	Description	Supported By
Modern form	Parameters declared inside the parentheses with both type and name	C89, C99, C++
Old-style / classic form	Parameters listed by name only inside parentheses; types declared between) and {	C89, C99 (designated obsolete); NOT C++

Comparison — modern vs old-style parameter declaration:

```

/* MODERN form (recommended): */
float f(int a, int b, char ch)
{
    /* ... */
}

/* OLD-STYLE form (obsolete): */
float f(a, b, ch) /* parameter NAMES only inside parentheses */
int a, b;        /* parameter TYPES declared here, between ) and { */
char ch;
{
    /* ... */
}
/* Note: old-style allows multiple parameters in one type declaration (int a, b;)
   but the modern form requires each to have its own type specifier (int a, int b) */

```

REMEMBER *The old-style form of parameter declaration is designated as OBSOLETE by Standard C and is NOT supported by C++. Never use it for new code.*

THE inline KEYWORD (C99)

C99 has added the keyword `inline`, which applies to functions. By preceding a function declaration with `inline`, you are telling the compiler to optimize calls to the function.

Aspect	Description
Purpose	Requests the compiler to optimize function calls, typically by expanding the function's code inline rather than performing a traditional call
Effect	Instead of a CALL instruction (which pushes/pops the stack), the function's code is replicated at each call site — eliminating call overhead
Compiler discretion	<code>inline</code> is a REQUEST, not a mandate — the compiler can ignore it if inlining is not possible or beneficial
Standard	Added by C99
C++ support	Also supported by C++ — the <code>inline</code> specifier is the same
Where described	Fully described in Part Two of the textbook

General form:

```

inline return-type function-name(parameter list)
{
    /* function body */
}

/* Example: */
inline int max(int a, int b)
{
    return (a > b) ? a : b;
}
/* The compiler may replace each call to max() with the expression itself,
   eliminating the function call overhead entirely. */

```

NOTE *The `inline` specifier is also supported by C++.*

CHAPTER 6 — QUICK REFERENCE SUMMARY

Function Form and Scope

Topic	Key Rule
General form	ret-type name(type p1, type p2, ...) { body }
Return type	Any type except array; use void if no value is returned
Parameters	Each must be declared individually with its own type — cannot share types with commas
Empty parameter list	Use (void) in C to explicitly state no parameters; () alone means old-style (no info given)
Function scope	Each function defines a block scope; code is private to the function
Cannot jump in	You cannot use goto to jump into the middle of another function
No nested functions	All functions have file scope — a function cannot be defined inside another function
Local variables	Exist only while the function executes; destroyed on exit; hold garbage unless initialized
Static locals	Retain value between calls; same storage as global but scope limited to the function

Function Arguments

Concept	Detail
Call by value	C's default — a copy of the argument is passed; the function cannot change the original
Call by reference	Achieved by passing a pointer (&var); the function uses *ptr to access and modify the original
Arrays as arguments	Array name (without index) passes the address of the first element — function can modify original data
argc	int: number of command line arguments (always ≥ 1 ; program name = argv[0])
argv	char *argv[]: array of pointers to command line argument strings
argv[0]	Always the program's own name
argv[t][i]	i-th character of the t-th command line argument

return Statement

Usage	Syntax / Notes
Return a value	return expression;
Return with no value (void)	return; (or fall through to closing }
Multiple returns	A function may contain any number of return statements
C89 — non-void, no value	Legal (but bad practice) — returns garbage value
C99/C++ — non-void, no value	Return MUST include a value; reaching } returns garbage (no syntax error, but a bug)
Return pointer type	Must explicitly declare pointer return type; never return pointer to a local variable
Return value discard	Legal — the caller does not have to use the returned value
Function on left of =	ILLEGAL: func() = value; will not compile

Prototypes and Parameters

Concept	Detail
Prototype form	type name(type p1, type p2, ...); — semicolon at end
Parameter names	Optional in prototype but recommended (enables better error messages)
Required in C?	No, but strongly encouraged
Required in C++?	YES — all functions must be prototyped
Library prototypes	Include the appropriate header file (e.g., <stdio.h> for printf())
Definition as prototype	If definition appears before first use, no separate prototype is needed
Variable length params	Use ... at end of parameter list: int func(int a, int b, ...);
Minimum param required	At least one named parameter must precede ...
Old-style declaration	type name(); — return type only, no parameter info — OBSOLETE
Old-style parameters	Names in (), types declared between) and { — OBSOLETE, not C++
Implicit int rule	Default to int if type omitted — supported by C89, REMOVED from C99
inline keyword	C99: request to expand function code inline; also in C++; compiler may ignore

Recursion

Concept	Detail
Definition	A function that calls itself
Stack usage	Each call allocates a new set of local variables and parameters on the stack
No new function copy	Only new values — the same function code is reused

Concept	Detail
Base case (REQUIRED)	A conditional that returns directly without a recursive call — prevents infinite recursion
Risk	Deep recursion can exhaust the stack — stack overrun crashes the program
Advantage	Cleaner code for inherently recursive problems (quicksort, tree traversal, AI)
Disadvantage	Often slower and more memory-intensive than iterative equivalents

— End of Chapter 6 Notes —